

INTRODUCTION TO THE **SPRING** FRAMEWORK

3 Major Problems with this code:

```
public class NotificationService {  
  
    private EmailService emailService = new EmailService();  
  
    public void sendNotification() {  
        System.out.println("Preparing notification...");  
        emailService.sendEmail("Your profile has been updated.");  
    }  
}
```

3 Major Problems with this code:

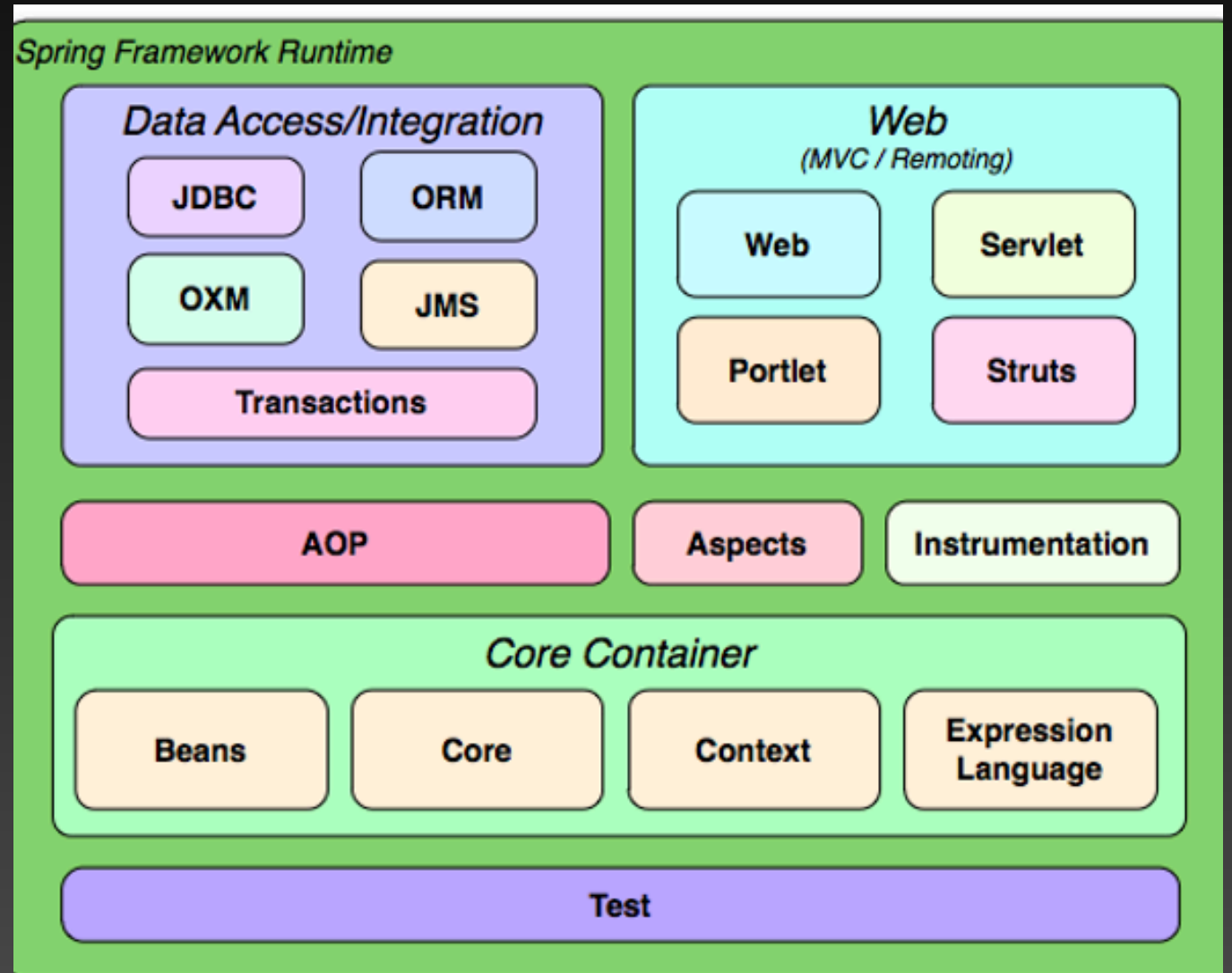
- Tight dependency — NotificationService is locked to EmailService.
- Hard to modify — Switching to SMS/WhatsApp requires changing code.
- Difficult to test — Cannot replace EmailService^P with a mock.
- Scattered Object Creation — Objects created everywhere using new, causing messy setup.

Tight coupling is only the start

- Older Java Enterprise applications faced many other challenges, such as:
- Poor and complicated JDBC / ORM handling
- Configuration files spread everywhere
- Difficult and lengthy deployment process
- Lots of repetitive boilerplate code
- Dependencies wired by hand, making testing and maintenance painful

Spring Framework Modules

Spring is built from multiple modules, each handling different application tasks. You can use only the modules you need, making projects flexible and easy to manage.



History of Spring Framework

Before Spring (Java EE / J2EE Problems)

- Heavy and complex architecture
- Hard to test and maintain
- Too many XML configurations
- Manual dependency wiring
- Complicated deployment process

Spring Framework Arrives

- 2004 — Spring Framework released (**Rod Johnson**)
- Introduced IoC & Dependency Injection
- Reduced configuration burden
- P • Simplified enterprise development
- 2004-2017, Spring Framework
- Evolved: Added Java-based Configuration
- Added support for Java 8
- Embraced microservices, reactive programming

History of SpringBoot Framework

- 2014 — Spring Boot launched
- Auto-configuration introduced
- Embedded servers (Tomcat / Jetty)
- Starter dependencies for faster setup
- Production features (Actuator, metrics, health)
- Major boost for microservices and cloud apps

Diff b/w Spring & SpringBoot Framework

Spring Framework

- Core feature: Dependency Injection
- Requires manual server setup
- Contains a lot of repetitive and boilerplate configuration
- No default in-memory database support
- Needs multiple dependencies manually added in pom.xml

Spring Boot

- Core feature: Auto-configuration
- Comes with built-in servers like Tomcat or Jetty
- Minimizes boilerplate by using starter packages
- Provides in-memory databases like H2 out of the box
- A full app can start using one starter dependency

Should you learn Spring First?

- DI (Dependency Injection): Spring provides objects automatically.
- Beans: Create & manage objects easily.
- ApplicationContext: Spring's control center.
- Spring MVC: Handle web requests & responses.
- Annotations: Use @Component, @Service, @Repository to mark classes.
- Configuration: Define beans with @Configuration & @Bean.

Now we can start
learning Spring
Boot 